

DBMS TASK .4

Order Management System

1. Create Database

```
CREATE DATABASE OrderManagementSystem;  
  
USE OrderManagementSystem;
```

2. Create Customers Table

```
CREATE TABLE Customers (  
    Customer_ID INT PRIMARY KEY,  
    Customer_Name VARCHAR(100) NOT NULL,  
    Email VARCHAR(100),  
    Phone VARCHAR(15)  
);
```

3. Create Products Table

```
CREATE TABLE Products (  
    Product_ID INT PRIMARY KEY,  
    Product_Name VARCHAR(100) NOT NULL,  
    Price DECIMAL(10,2) NOT NULL  
);
```

4. Create Orders Table

The Orders table stores the main information about each customer order.

```
CREATE TABLE Orders (  
    Order_ID INT PRIMARY KEY,  
    Customer_ID INT NOT NULL,  
    Order_Date DATE NOT NULL,  
    Total_Amount DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (Customer_ID) REFERENCES Customers(Customer_ID)  
);
```

5. Create Order_Details Table

The Order_Details table stores the products and quantities included in each order.

```
CREATE TABLE Order_Details (  
    Order_Detail_ID INT PRIMARY KEY,  
    Order_ID INT NOT NULL,  
    Product_ID INT NOT NULL,  
    Quantity INT NOT NULL,  
    Unit_Price DECIMAL(10,2) NOT NULL,  
    Subtotal DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),
```

```
FOREIGN KEY (Product_ID) REFERENCES Products(Product_ID)
);
```

6. Insert Customer Records

```
INSERT INTO Customers (Customer_ID, Customer_Name, Email, Phone)
VALUES
(1, 'Arun Kumar', 'arun@gmail.com', '9876543210'),
(2, 'Priya Sharma', 'priya@gmail.com', '9876543211'),
(3, 'Rahul Raj', 'rahul@gmail.com', '9876543212'),
(4, 'Sneha Devi', 'sneha@gmail.com', '9876543213');
```

7. Insert Product Records

```
INSERT INTO Products (Product_ID, Product_Name, Price)
VALUES
(101, 'Laptop', 55000.00),
(102, 'Mouse', 800.00),
(103, 'Keyboard', 1500.00),
(104, 'Headphones', 2500.00),
(105, 'Monitor', 12000.00);
```

8. Insert Orders

```
INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount)
VALUES
(1001, 1, '2026-08-01', 55800.00),
(1002, 2, '2026-08-05', 4000.00),
(1003, 1, '2026-08-10', 14500.00),
(1004, 3, '2026-08-12', 2500.00);
```

9. Insert Order Details

```
INSERT INTO Order_Details
(Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)
VALUES
(1, 1001, 101, 1, 55000.00, 55000.00),
(2, 1001, 102, 1, 800.00, 800.00),
(3, 1002, 103, 1, 1500.00, 1500.00),
(4, 1002, 104, 1, 2500.00, 2500.00),
(5, 1003, 105, 1, 12000.00, 12000.00),
(6, 1003, 103, 1, 1500.00, 1500.00),
(7, 1004, 104, 1, 2500.00, 2500.00);
```

10. Order Insertion Operation

A new order can be inserted using the following query:

```
INSERT INTO Orders
(Order_ID, Customer_ID, Order_Date, Total_Amount)
VALUES
(1005, 4, '2026-08-15', 6700.00);
```

Then insert the corresponding order details:

```
INSERT INTO Order_Details
(Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)
VALUES
(8, 1005, 102, 2, 800.00, 1600.00),
(9, 1005, 104, 2, 2500.00, 5000.00);
```

11. Order Modification Operation

The total amount of an order can be modified using:

```
UPDATE Orders
SET Total_Amount = 7200.00
WHERE Order_ID = 1005;
```

The quantity of an ordered product can also be modified:

```
UPDATE Order_Details
SET Quantity = 3,
    Subtotal = 2400.00
WHERE Order_Detail_ID = 8;
```

12. Delete an Order

First delete the order details and then delete the order:

```
DELETE FROM Order_Details
WHERE Order_ID = 1005;
```

```
DELETE FROM Orders
WHERE Order_ID = 1005;
```

13. Display All Orders

```
SELECT * FROM Orders;
```

14. Display Order Details

```
SELECT * FROM Order_Details;
```

15. Customer Order History Report

The following query generates the complete order history of all customers:

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    o.Order_ID,
    o.Order_Date,
    p.Product_Name,
    od.Quantity,
    od.Unit_Price,
    od.Subtotal,
    o.Total_Amount
```

```

FROM Customers c
JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
JOIN Order_Details od
    ON o.Order_ID = od.Order_ID
JOIN Products p
    ON od.Product_ID = p.Product_ID
ORDER BY c.Customer_ID, o.Order_Date;

```

16. Order History for a Particular Customer

For example, to display the order history of customer 1:

```

SELECT
    c.Customer_Name,
    o.Order_ID,
    o.Order_Date,
    p.Product_Name,
    od.Quantity,
    od.Subtotal
FROM Customers c
JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
JOIN Order_Details od
    ON o.Order_ID = od.Order_ID
JOIN Products p
    ON od.Product_ID = p.Product_ID
WHERE c.Customer_ID = 1
ORDER BY o.Order_Date;

```

17. Calculate Total Amount of Each Order

```

SELECT
    o.Order_ID,
    c.Customer_Name,
    o.Order_Date,
    SUM(od.Subtotal) AS Calculated_Total
FROM Orders o
JOIN Customers c
    ON o.Customer_ID = c.Customer_ID
JOIN Order_Details od
    ON o.Order_ID = od.Order_ID
GROUP BY o.Order_ID, c.Customer_Name, o.Order_Date;

```

18. Find Total Amount Spent by Each Customer

```

SELECT
    c.Customer_ID,
    c.Customer_Name,
    SUM(o.Total_Amount) AS Total_Spent
FROM Customers c
JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Spent DESC;

```

19. Find Number of Orders Made by Each Customer

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    COUNT(o.Order_ID) AS Number_of_Orders
FROM Customers c
LEFT JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name;
```

20. Conclusion

The Order Management System successfully manages customer orders using the Customers, Products, Orders, and Order_Details tables. It supports order insertion, modification, deletion, product quantity management, total amount calculation, and customer order history reports. The use of primary keys and foreign keys maintains data integrity and establishes relationships between customers, orders, and products.